TS-Com Project Documentation Suite

Edge Computing Network
Course *CP352005 Networks - 4-Week Sprint with 5-Member Role-Assigned Team*

Member: ปิยชญานินท์ โทนะพันธ์ 673380049-4
วิภู หากันได้ 673380291-7
ฐิติศักดิ์ ภูขยัน 673380035-5
ศุภเชฏฐ์ วอหล้า 673380063-0
ชาคริต ภักมี 673380265-8

File 1: architecture_spec.md

```
None
# File 1: architecture_spec.md

# Architecture Specification: Decentralized, Edge-First Network v1.0
**Architectural Review Document - CP352005 Networks (4-Week Sprint
Project)**

## Document Control
| Version | Date | Author | Role | Changes |
| :--- | :--- | :--- | :--- | :--- |
| v1.0 | 28/02/2026 | [ชื่อ Architect] | Architect | Initial architectural
review |

## Team Roles
| Role Name | Assigned To | Responsibilities |
| :--- | :--- | :--- |
| **Architect** | [ชื่อ] | ออกแบบระบบภาพรวม, กำหนดโปรโตคอล, ร่างสัญญาการเชื่อมต่อ
(Interface contracts) |
| **Engineer** | [ชื่อ] | เขียนโค้ดโปรโตคอลหลัก, พัฒนาอัลกอริทึม P2P และ CRDTs, สร้างระบบ
จำลอง |
| **Specialist** | [ชื่อ] | วิจัยและออกแบบ Zero-Trust Security, กำหนดลอจิกของ
Eventual Consistency |
| **DevOps** | [ชื่อ] | จัดเตรียมสภาพแวดล้อม (Mininet/Docker), จัดการ CI/CD, ควบคุม
เวอร์ชัน |
| **Tester/QA** | [ชื่อ] | วางแผนการทดสอบ, สร้าง Test cases สำหรับสถานการณ์ Network
Partition |

---

## Part 1: Executive Summary
```

1.1 Project Vision
โครงการนี้มีจุดมุ่งหมายเพื่อจำลองและพิสูจน์แนวคิดกระบวนทัศน์ใหม่ของเครือข่ายที่เรียกว่า **Decentralized, Edge-First Network** ซึ่งออกแบบมาเพื่อก้าวข้ามข้อจำกัดของระบบคลาวด์แบบรวมศูนย์ (Cloud-centric) ในด้านความหน่วงและการขยายขนาด ระบบอาศัยการประสานงานแบบเพียร์ทูเพียร์ (P2P) ระหว่างโหนดขอบข่ายที่เป็นอิสระ (Autonomous Edge Nodes) เพื่อให้ระบบสามารถอยู่รอดได้แม้ถูกตัดขาดจากอินเทอร์เน็ตหลัก

1.2 Educational Objectives
* ประยุกต์ใช้แนวคิดระบบกระจายศูนย์ (Distributed Systems) ในสถานการณ์เครือข่ายจำลอง
* ออกแบบและทดสอบการรักษาสถานะข้อมูลแบบ Eventual Consistency โดยใช้โครงสร้างข้อมูล CRDTs
* จำลองสถาปัตยกรรมความปลอดภัยแบบ Zero-Trust (P2P mTLS)
* ฝึกฝนการทำงานแบบ Agile/Sprint และการจัดทำเอกสารทางสถาปัตยกรรมระบบ

1.3 Scope and Constraints
Aspect	In Scope	Out of Scope
Architecture	Tiered Edge Network (Sensor -> Edge Node -> Cloud)	Physical hardware deployment
Simulation	การจำลองด้วย Python / Mininet / Docker	การนำไปใช้ใน Production จริง
Consistency	Eventual Consistency (CRDTs)	Strict Consistency (เช่น Paxos)
Security	P2P Identity (SPIFFE/mTLS)	ระบบป้องกัน DDoS เต็มรูปแบบ

Part 2: Architectural Review

2.1 Architecture Overview
เลเยอร์ (Layer)	เทคโนโลยี / อัลกอริทึม	หน้าที่หลัก
Application Layer	Edge Analytics / Real-time Telemetry	ประมวลผลและตัดสินใจทันทีที่ขอบเครือข่าย
Data Layer	CRDTs (Conflict-free Replicated Data Types)	รักษาสถานะข้อมูลให้ตรงกันโดยไม่เกิดข้อขัดแย้ง
Coordination Layer	P2P Gossip Protocol	กระจายข้อมูลสถานะระหว่างโหนดเพื่อนบ้าน
Security Layer	Zero-Trust / SPIFFE	ยืนยันตัวตนแบบกระจายศูนย์ (Decentralized Identity)
Network Layer	IPv6 / Mesh Routing	กำหนดเส้นทางแบบเพียร์ทูเพียร์

2.2 Core Component Review

2.2.1 Data Consistency (CRDTs)
* **สถานะการออกแบบ:** **อนุมัติ (Approved)**

```
* **รายละเอียด:** ใช้ State-based CRDTs เพื่อจัดการข้อมูลสถานะ (เช่น พิกัด, เซนเซอร์) ช่วยให้โหนดสามารถอัปเดตข้อมูลได้เองเมื่อเกิด Network Partition และสามารถรวมข้อมูล (Merge) ได้อัตโนมัติเมื่อการเชื่อมต่อกลับมา
* **ความเสี่ยง:** ขนาดของ Payload อาจใหญ่ขึ้นตามระยะเวลาที่ขาดการเชื่อมต่อ (จำเป็นต้องมีกลไก Garbage Collection)

#### 2.2.2 Peer-to-Peer Coordination (Gossip Protocol)
* **สถานะการออกแบบ:** **อนุมัติแบบมีเงื่อนไข (Conditional Approval)**
* **รายละเอียด:** โหนดจะแลกเปลี่ยนข้อมูลกับเพื่อนบ้านแบบสุ่มเพื่อแพร่กระจายสถานะของเครือข่าย (Epidemic routing)
* **เงื่อนไข:** Engineer ต้องกำหนดพารามิเตอร์ของ `Gossip Interval` (เช่น ทุกๆ 50ms) และ `Fanout size` เพื่อป้องกันพายุข้อมูล (Broadcast storm) ภายใน Sprint แรก

#### 2.2.3 Zero-Trust Security (Decentralized Identity)
* **สถานะการออกแบบ:** **อนุมัติ (Approved)**
* **รายละเอียด:** โหนดทุกตัวต้องยืนยันตัวตนก่อนพูดคุยกันผ่าน mTLS (Mutual TLS) โดยใช้ใบรับรองอายุสั้น (Short-lived certificates) จากระบบที่จำลองแนวคิดของ SPIRE

### 2.3 Interface Contracts
ตัวอย่างการสื่อสารระหว่างโหนด (Edge-to-Edge):
```python
# Node A sends a gossip state update to Node B
edge_node_b.receive_gossip({
    "node_id": "edge-001",
    "timestamp": 1678886400,
    "crdt_payload": {"sensor_x": 45.2, "vector_clock": [2, 0, 1]},
    "signature": "hash_xyz..."
})
```

2.4 Non-Functional Requirements

Requirement	Target	Validation Method
Latency	< 10ms ระหว่าง Edge Nodes	Simulation metrics (Ping/Processing time)
Survivability	ทำงานได้ 100% เมื่อตัด Uplink	Network Partition Test Scenario
Consistency	Converge ภายใน 5 วินาทีหลังเน็ตกลับมา	CRDT Merge Logging
```

Part 3: Architecture Decisions Log

- **Decision 1: เลือกใช้ CRDTs แทน Consensus (Raft)**
  - วันที่: 28/02/2026
  - เหตุผล: Raft กินทรัพยากรสูงและต้องการการเชื่อมต่อที่เสถียรเพื่อเลือกผู้นำ (Leader Election) CRDTs ตอบโจทย์การทำงานแบบ Ad-hoc และ Eventual Consistency ได้ดีกว่าสำหรับ Edge
  - สถานะ: อนุมัติ
- **Decision 2: การจำลองด้วย Mininet-WiFi / Docker**
  - วันที่: 28/02/2026
  - เหตุผล: จำลอง Topology แบบ Mesh และการขาดหายของสัญญาณ (Partition) ได้สมจริง
  - สถานะ: อนุมัติ

Role	Name	Signature	Date	Comments
Architect	[ชื่อ]			อนุมัติให้เริ่มดำเนินการ
Engineer	[ชื่อ]			รับทราบข้อกำหนด
Specialist	[ชื่อ]			
DevOps	[ชื่อ]			
Tester/QA	[ชื่อ]			

None

```
# Implementation and Sprint Plan v1.0
**Project: Decentralized, Edge-First Network - CP352005 Networks**

## Document Control
| Version | Date | Author | Role |
| :--- | :--- | :--- | :--- |
| v1.0 | 28/02/2026 | [ชื่อ] | [Role] |

## Part 1: Sprint Overview
ระยะเวลาการทำงานทั้งหมด 4 สัปดาห์ (4-Week Sprint) ภายใต้กระบวนการ Agile โดยมีการติดตามความคืบหน้า (Standups) และแก้ไขปัญหาที่บล็อกการทำงาน (Blockers) อย่างสม่ำเสมอ

---

## Part 2: Weekly Execution Plan

### Week 1: Foundation & Architecture Setup
*เป้าหมาย: อนุมัติสถาปัตยกรรมระบบและเตรียมสภาพแวดล้อมการพัฒนาให้พร้อม*
* **Architect:** จัดทำและแก้ไข `architecture_spec.md` ให้เสร็จสมบูรณ์, อนุมัติ Interface contracts
* **DevOps:** จัดเตรียม Repository (GitHub/GitLab), สร้าง Dockerfile เบื้องต้นสำหรับ Edge Node และตั้งค่า Mininet
* **Engineer:** วางโครงสร้างโค้ด (Skeleton code) ด้วย Python สำหรับ P2P Node
* **Specialist:** ศึกษางานวิจัยและร่างกระบวนการทำงานของ CRDTs สำหรับ Python
* **Tester/QA:** ร่างกลยุทธ์การทดสอบ (Test Strategy) สำหรับ P2P Network

### Week 2: Core Component Implementation
*เป้าหมาย: พัฒนาฟีเจอร์หลัก (P2P, CRDTs, Zero-Trust) ให้ใช้งานได้เบื้องต้น*
* **Engineer:** เขียนโค้ดระบบ Gossip Protocol สำหรับกระจายข้อมูล และนำ CRDTs ไปใช้กับชุดข้อมูล
* **Specialist:** เขียนลอจิกจำลองกลไกการออก Certificate ระยะสั้น (SPIRE mock)
* **Architect:** คอยให้คำแนะนำด้าน Design Pattern และติดตาม Technical Debt
* **DevOps:** นำโค้ดเข้าสู่ CI/CD Pipeline (Build automation)
* **Tester/QA:** เขียน Unit Tests ให้กับส่วนลอจิกของ CRDTs (ตรวจสอบการ Merge ว่าไม่พัง)

### Week 3: Integration & Network Simulation
*เป้าหมาย: รวมโค้ดเข้าด้วยกันและรันบนระบบจำลองเครือข่าย*
* **DevOps:** ประสานงานการนำ Node เข้าไปรันใน Mininet / Docker network พร้อมกำหนดค่า Topology
* **Engineer & Specialist:** ทดสอบการส่งข้อมูลข้าม Node จริงๆ ภายใน Network เสมือน และแก้ไขบั๊ก (Integration fixing)
* **Architect:** ตรวจสอบและประเมินประสิทธิภาพ (Performance) ว่า Latency เป็นไปตามเป้าหมายหรือไม่
* **Tester/QA:** พัฒนาสคริปต์สำหรับจำลองสถานการณ์ Network Partition (ตัดการเชื่อมต่อระหว่าง Node บางตัว)

### Week 4: Testing, Validation & Demo Preparation
```

เป้าหมาย: ตรวจสอบคุณภาพขั้นสุดท้าย, เตรียม Mockup และเอกสารนำเสนอ
* **Tester/QA:** รัน Test case ทั้งหมด (เป้าหมาย Test coverage > 80%), ตรวจสอบและรายงานผล Quality metrics
* **Engineer & DevOps:** แก้ไขบั๊กสุดท้าย (Bug fixes) และเตรียม Deployment scripts สำหรับวันพรีเซนต์
* **Architect & Specialist:** รวบรวมข้อมูล, สรุปผลการทดลอง, และจัดเตรียม Project Pitch & Presentation (เวลา 14.5 นาทีตามโจทย์)
* **All Team:** ซ้อมนำเสนอ และเตรียมตัวสำหรับ Group/Teacher Feedback

---

Part 3: Issue Escalation Process (Technical Block)
หากพบปัญหาทางเทคนิคที่ทำให้งานไม่คืบหน้า ให้ดำเนินการดังนี้:
1. พยายามแก้ปัญหาด้วยตัวเอง ไม่เกิน 2 ชั่วโมง (2-hour investigation limit)
2. แจ้งและขอความช่วยเหลือจากทีมในกลุ่ม
3. ค้นหา Workaround (วิธีเลี่ยงปัญหา) เพื่อให้งานส่วนอื่นเดินหน้าต่อได้
4. ปรึกษาและขอคำแนะนำจากอาจารย์ผู้สอน (Instructor)

Part 4: Success Criteria Sign-off

Criteria	Target	Current	Owner	Status
Architecture approved	Week 1		Architect	⬜ Pending
All components implemented	Week 2		Engineer	⬜ Pending
Integration complete	Week 3		DevOps	⬜ Pending
Tests passing (>80% coverage)	Week 4		Tester	⬜ Pending
Demo & Mockup ready	Week 4		All	⬜ Pending
Documentation complete	Week 4		All	⬜ Pending
Presentation prepared	Week 4		All	⬜ Pending

Part 5: Implementation Plan Approval
ทีมงานทุกคนรับทราบและเห็นชอบกับแผนการดำเนินงานสำหรับโครงงานวิชา CP352005 Networks ฉบับนี้

Role	Name	Signature	Date
Architect	[Name]		
Engineer	[Name]		
Specialist	[Name]		
DevOps	[Name]		
Tester/QA	[Name]		

```
Week 1        Week 2        Week 3        Week 4  ┌──────────────┐
┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │
Architecture│→│ Protocols  │→│ Integration │→│ Final Demo  │ │ Design     │ │
Implementation│ │ & Testing  │ │ & Delivery  │ └──────────────┘
└──────────────┘ └──────────────┘ └──────────────┘ ↓          ↓
↓            ↓ CAAP Spec → CAAP Code → CAAP Test → CAAP Integrated TRP Spec →
TRP Code  → TRP Test  → TRP Integrated PPP Spec  → PPP Code  → PPP Test  → PPP
Integrated Sim Setup → Sim Dev   → Sim Test  → Sim Final
```

```
None

# Sprint Plan: Decentralized, Edge-First Network
**Project: CP352005 Networks - 4-Week Sprint**

## 1. Sprint Overview
* **Sprint Name:** Edge-First Network Simulation (v1.0)
* **Duration:** 4 Weeks (4 Sprints)
* **Sprint Goal:** จำลองและพิสูจน์แนวคิดการทำงานของโหนดขอบข่ายแบบกระจายศูนย์ (Decentralized Edge Nodes) ที่สามารถสื่อสารกันแบบ P2P รักษาความสอดคล้องของข้อมูลด้วย CRDTs และอยู่รอดได้แม้เครือข่ายหลักล่ม (Network Partition)

## 2. Sprint Ceremonies (พิธีกรรมประจำสปริ้นท์)
* **Sprint Planning (วางแผน):** ทุกวันจันทร์ (สัปดาห์ละ 1 ครั้ง) ใช้เวลา 1 ชั่วโมง เพื่อดึงงานจาก Backlog มากำหนดเป้าหมายรายสัปดาห์
* **Daily Standup (อัปเดตงานรายวัน):** ทุกวัน (15 นาที) แต่ละคนตอบคำถาม 3 ข้อ:
  1. เมื่อวานทำอะไร?
  2. วันนี้จะทำอะไร?
  3. มีปัญหา/บล็อกเกอร์ (Blocker) อะไรไหม?
* **Sprint Review (ตรวจรับงาน):** ทุกวันศุกร์ (30 นาที) เพื่อนำเสนอความคืบหน้าของโค้ด/เอกสาร ให้กับทีมและเตรียมรวมโค้ด
* **Sprint Retrospective (ประเมินผล):** ทุกวันศุกร์หลัง Review (15 นาที) เพื่อคุยว่าสิ่งใดทำได้ดี สิ่งใดต้องปรับปรุงในสัปดาห์ถัดไป

---

## 3. Sprint Backlog & Timeline (แผนงานรายสัปดาห์)

### Sprint 1: Architecture & Foundation (สัปดาห์ที่ 1)
**Sprint Goal:** กำหนดสถาปัตยกรรมให้ชัดเจนและเตรียมสภาพแวดล้อมให้พร้อมเริ่มเขียนโค้ด
* **[Task 1]** (Architect) ร่างและอนุมัติ `architecture_spec.md` และกำหนด Data Payload format
* **[Task 2]** (DevOps) ตั้งค่า Git Repository, สร้าง Dockerfile และโครงสร้าง Mininet เสมือน
* **[Task 3]** (Engineer) สร้าง Skeleton Code ภาษา Python สำหรับ Edge Node เบื้องต้น
* **[Task 4]** (Specialist) สรุปขั้นตอนการทำ Eventual Consistency ด้วย CRDTs เป็น Flowchart
```

```
* **[Task 5]** (Tester/QA) ร่าง `test_plan.md` สำหรับการทดสอบ P2P และ Network
Partition

### Sprint 2: Core Components Development (สัปดาห์ที่ 2)
**Sprint Goal:** พัฒนาฟีเจอร์หลักให้โหนดสามารถพูดคุยกันและอัปเดตสถานะได้
* **[Task 1]** (Engineer) เขียนลอจิก P2P Gossip Protocol ให้โหนดส่งข้อมูลหากันได้
* **[Task 2]** (Engineer) เขียนระบบ CRDTs State Merge ลงในโหนด
* **[Task 3]** (Specialist) พัฒนาโมดูล Zero-Trust (mTLS/SPIFFE Mockup) สำหรับ
ยืนยันตัวตนโหนด
* **[Task 4]** (DevOps) นำโค้ดเข้าสู่ระบบ CI/CD เพื่อให้รัน Build อัตโนมัติ
* **[Task 5]** (Architect) ตรวจสอบ Code Quality และให้คำแนะนำเรื่อง Design Pattern
กับทีม
* **[Task 6]** (Tester/QA) สร้าง Unit Test สำหรับทดสอบการ Merge ข้อมูลของ CRDTs

### Sprint 3: Integration & Network Partitioning (สัปดาห์ที่ 3)
**Sprint Goal:** รวมระบบทั้งหมดเข้าด้วยกัน และทดสอบในสภาวะที่เครือข่ายล่ม
* **[Task 1]** (DevOps) นำโหนดทั้งหมดไปรันบน Mininet พร้อมกำหนด Topology แบบ Mesh
* **[Task 2]** (Engineer & Specialist) ทดสอบส่งข้อมูลจริงบน Mininet และแก้บั๊กการเชื่อม
ต่อ (Integration Fixes)
* **[Task 3]** (Tester/QA) รันสคริปต์ตัดการเชื่อมต่อ (Network Partition Simulation)
และบันทึกผล
* **[Task 4]** (Architect) เก็บ Log ประสิทธิภาพการทำงาน (Latency) และตรวจดูว่าต่ำกว่า
10ms หรือไม่

### Sprint 4: Polish, QA & Pitch Preparation (สัปดาห์ที่ 4)
**Sprint Goal:** ตรวจสอบคุณภาพขั้นสุดท้ายและเตรียมพรีเซนต์
* **[Task 1]** (Tester/QA) รัน End-to-End Test (เป้าหมาย Pass > 80%) และออก
รายงาน QA
* **[Task 2]** (Engineer & DevOps) ปรับจูนประสิทธิภาพสุดท้าย (Performance
Optimization) และ Freeze Code
* **[Task 3]** (Architect, Specialist) จัดทำเอกสาร Project Pitch (ปัญหา, วิธีแก้,
ผลลัพธ์)
* **[Task 4]** (All) ซ้อมนำเสนอ 14.5 นาที แบ่งตามบทบาท
(Architect/Engineer/Specialist ช่วงแรก -> DevOps ช่วงสอง -> Tester/QA ช่วงท้าย)
* **[Task 5]** (All) เตรียมระบบ Mockup ให้อยู่ในสถานะพร้อม Demo ต่อหน้าอาจารย์

---

## 4. Definition of Done (DoD)
งานในแต่ละ Task จะถือว่า **"เสร็จสมบูรณ์"** ก็ต่อเมื่อ:
1. โค้ดถูกเขียนและคอมมิต (Commit) ลงใน Branch ที่ถูกต้อง
2. โค้ดผ่านการทดสอบ Unit Test เบื้องต้น (ไม่พังเมื่อรัน)
3. มีการเขียนคอมเมนต์อธิบายโค้ด (Documentation)
4. ได้รับการอนุมัติ (Code Review) จากทีมอย่างน้อย 1 คน (มักจะเป็น Architect หรือ DevOps)
5. หากเป็นงานเอกสาร ต้องได้รับการลงนามเห็นชอบจากทีมครบถ้วน

---
```

```
## 5. Team Sign-off
เราขอยืนยันว่าจะปฏิบัติตาม Sprint Plan นี้เพื่อส่งมอบงานให้ทันกำหนด:
* **Architect:** _____
* **Engineer:** _____
* **Specialist:** _____
* **DevOps:** _____
* **Tester/QA:** _____
```